

Feature 1: Designated Initializers

This is the main reason the project requires C++20. The build enforces CMAKE_CXX_STANDARD 20 and target_compile_features(vidchopper_core PUBLIC cxx_std_20), and designated initializers appear across the core library, Qt services, UI model, and tests.

Where it appears

Eight files use C++20 aggregate designators: `src/core/chapter_plan.cpp`, `src/core/command_builder.cpp`, `src/qt/services/export_coordinator.cpp`, `src/qt/services/ffprobe_service.cpp`, `src/qt/ui/chapter_table_model.cpp`, and three tests. These sites construct data-model structs such as `ChapterSegment`, `ResolvedEncoder`, `VideoProbeResult`, and `PendingExport`.

Why it matters here

VidChopper is structured around small, plain structs with defaulted members. Named field initialization lets each call site set only the fields it cares about while inheriting defaults for the rest, without depending on declaration order.

[Core library](#)[Qt service layer](#)[Tests mirror production data](#)

IN-CONTEXT C++20 EXAMPLE

```
src/core/command_builder.cpp:49-68
if (settings.encoder_kind == EncoderKind::HvcNvenc) {
    return ResolvedEncoder {
        .kind = EncoderKind::HvcNvenc,
        .video_codec = "hevc_nvenc",
        .arguments = {"-preset", settings.nvenc_preset, "-cq",
                     std::to_string(settings.nvenc_cq), "-rc", "vbr_hq"},
    };
}

return ResolvedEncoder {
    .kind = EncoderKind::X264,
    .video_codec = "libx264",
    .arguments = {"-preset", settings.x264_preset, "-crf",
                 std::to_string(settings.x264_crf)},
};
```

MORE PRODUCTION USES

```
src/core/chapter_plan.cpp:32-36
chapters.push_back(ChapterSegment {
    .name = "Chapter " + std::to_string(index + 1),
    .start_ms = start_ms,
    .end_ms = end_ms,
});

src/qt/services/ffprobe_service.cpp:129-132
return VideoProbeResult {
    .success = metadata.duration_ms > 0,
    .error_message = metadata.duration_ms > 0 ? QString {} : ...,
    .metadata = std::move(metadata),
};
```

C++17 EQUIVALENT

SAME INTENT WITHOUT C++20 DESIGNATORS

```
ResolvedEncoder resolved {};
resolved.kind = EncoderKind::HvcNvenc;
resolved.video_codec = "hevc_nvenc";
resolved.arguments = {
    "-preset", settings.nvenc_preset,
    "-cq", std::to_string(settings.nvenc_cq),
    "-rc", "vbr_hq"
};
return resolved;
```

WHY THIS VERSION WAS CHOSEN

- It makes object construction self-documenting at the call site.
- It avoids brittle positional aggregate initialization for larger structs.
- It lets omitted members keep their struct defaults cleanly.
- It keeps tests and production code using the same data-construction style.

DIFFERENCE IT MAKES

- Performance is effectively neutral versus a default-construct-then-assign sequence.
- The codebase benefit is large: many model-construction sites become shorter and safer to edit.
- Field-order mistakes are easier to spot because every populated member is named explicitly.

Feature 2: Ranges Algorithm Overload

The core library uses `std::ranges::transform` in the path-normalization logic that selects a safe output extension. This is a real C++20 dependency in shipped code, not just a test convenience.

Where it appears

One occurrence in `src/core/command_builder.cpp` inside `safe_source_extension`. The function lowercases the extension in place before deciding whether the source container is safe to preserve.

Why it matters here

The code works on a whole string object, not on a pair of iterators stored elsewhere. The ranges overload expresses that directly and keeps the transformation local to the range being mutated.

IN-CONTEXT C++20 EXAMPLE

```

SRC/CORE/COMMAND_BUILDER.CPP:10-27
auto safe_source_extension(const VideoMetadata& metadata)
-> std::string {
    auto extension = metadata.source_extension;
    if (extension.empty()) {
        extension = metadata.source_path.extension().string();
    }

    if (extension.empty()) {
        return ".mp4";
    }

    std::ranges::transform(extension, extension.begin(),
        [](const unsigned char character) {
            return static_cast<char>(std::tolower(character));
        });

    if (extension != ".mp4" && extension != ".mkv" &&
        extension != ".mov") {
        return ".mp4";
    }
    return extension;
}

```

C++17 EQUIVALENT

```

USING ITERATOR PAIRS
std::transform(
    extension.begin(),
    extension.end(),
    extension.begin(),
    [](const unsigned char character) {
        return static_cast<char>(std::tolower(character));
    });

```

WHY THIS VERSION WAS CHOSEN

- The algorithm takes the range object directly, so the code repeats less iterator boilerplate.
- It reads like 'transform this string' instead of 'transform begin/end into begin'.
- It matches the rest of the codebase's operate-on-whole-objects style.

DIFFERENCE IT MAKES

- Runtime behavior is essentially the same as `std::transform`: in-place lowercase conversion, no extra allocation.
- The gain is mainly maintainability and intent clarity, not raw speed.
- Even though this is only one site, it still makes the core target require a C++20 library implementation.

Feature 3: C++20 String Suffix API

The test suite uses the C++20 convenience member `ends_with` to assert suffix behavior in command generation. This is not the main reason the product needs C++20, but it is one of the project's actual post-17 features.

Where it appears

One occurrence in `tests/test_command_builder.cpp`. The test verifies that source container mode preserves a supported source extension rather than forcing `.mp4`.

Scope note

This feature is test-only. If it were the only C++20 usage, the application itself could still be lowered to C++17 with test rewrites. The production code cannot, because designated initializers and ranges are used in shipped targets.

IN-CONTEXT C++20 EXAMPLE

```
TESTS/TEST_COMMAND_BUILDER.CPP:36-43
const auto command = build_ffmpeg_command(
    metadata, chapter, output_path, settings,
    EncoderEnvironment {});

test_support::expect_true(
    command.back().ends_with(".mov"),
    "source container mode should preserve common source extension"
);
```

C++17 EQUIVALENT

```
MANUAL SUFFIX CHECK
const auto& output = command.back();
const auto has_mov_suffix =
    output.size() >= 4 &&
    output.compare(output.size() - 4, 4, ".mov") == 0;

test_support::expect_true(
    has_mov_suffix,
    "source container mode should preserve common source extension"
);
```

WHY THIS VERSION WAS CHOSEN

- It states the intent of the assertion directly.
- It avoids repeated boundary arithmetic in test code.
- It keeps the test focused on behavior instead of string-index bookkeeping.

DIFFERENCE IT MAKES

- Performance difference is negligible and irrelevant in a unit test.
- The real benefit is correctness and readability: no risk of forgetting the length guard or getting the offset wrong.
- Because the use is isolated to tests, its impact on the overall codebase is small compared with designated initializers.

EXCLUSIONS FROM THE AUDIT

- The scan deliberately excluded framework APIs that look similar, such as `QString::contains` and `QString::startsWith`.
- Those are Qt features, not reasons to require C++20.